Smartphone BI Agent — Full System Architecture

Next.js Frontend · FastAPI Backend · ML Inference Pipeline · Model Store · Training Pipeline · External APIs

USER LAYER

Web Browser — HTTP / JSON

Business Analyst
Enters phone specs and explores BI insights via browser

Web Browser (Chrome / Edge / Firefox)
localhost:3000 — REST over HTTP — JSON payloads

HTTP/JSON

PRESENTATION LAYER

Next.js 16 · React 19 · TypeScript · Tailwind CSS · shadcn/ui · Recharts · port 3000

- bi-agent-app.tsx (App Shell)
- specs-form.tsx (22-field PhoneSpec Form)
- dashboard.tsx (Main Result Panel)
- theme-provider.tsx + logo.tsx
- app/page.tsx + layout.tsx
- demand-chart.tsx (6-Month Bar Chart)
- pmf-gauge.tsx (PMF Gauge + GO/NO-GO)
- competitor-radar.tsx (Radar / Spider Chart)
- price-optimizer.tsx (Profit Curve Chart)
- donut-chart.tsx (Demographics Donut)
- loyalty-diverging.tsx (RFM Diverging Bar)
- news-panel.tsx (Article Cards + Tags)
- ui/card · badge · tabs · button · slider · select
- ui/separator · label (shadcn/ui base components)

API GATEWAY — backend/main.py

FastAPI (Python) · Pydantic v2 · CORSMiddleware · Uvicorn · port 8000

- **GET /api/health** — Returns server status
- **POST /api/analyze** — Full BI analysis PhoneSpec -> JSON
- **POST /api/competitors** — k-NN radar data + spec insights
- **POST /api/price-optimize** — 14-point sweep profit curve
- **POST /api/news** — Parallel web search ThreadPoolExecutor
- **Pydantic PhoneSpec** — 22 validated fields

CORSMiddleware — allow_origins: localhost:3000 · allow_methods: * · allow_headers: *

ORCHESTRATION LAYER — src/agent.py

Unified BI Pipeline — Sequential + CLV

analyze() entry → Stage 1/2 Market Research → calls analyze() Stage 3 Segmentation → Stage 4 Demand Forecast → Stage 5 PMF → Stage 6 estimate_clv() → config.py add_engineered_features() → Result JSON

ML INFERENCE LAYER

scikit-learn · XGBoost · joblib · numpy · pandas · CPU-only · No GPU required

Pillar 1 — Market Research
src/pillar1_market_research.py
— (A) SEGMENT CLASSIFIER —
XGBoostClassifier
n_estimators=300 max_depth=6
learning_rate=0.1 subsample=0.9
Output: Budget/Mid/Premium/Flagship
Accuracy ~99% (group-split)
— (B) SIMILARITY ENGINE —
NearestNeighbors k=5
StandardScaler-normalised specs
Output: 5 similar phones + dist

Pillar 2 — Demand Forecast
src/pillar2_demand.py
— DEMAND REGRESSOR —
XGBoostRegressor
Target: log1p(units_sold_monthly)
Features: NUM_SPECS + TIME_CONTEXT + ENGINEERED + REGION
brand_demand_prior (train fold)
Horizon: 6 months
Output: [month_1 .. month_6 units]
R^2 approx 0.85 (group-split)

Pillar 3 — Segmentation
src/pillar3_segmentation.py
— k-NN RETRIEVAL —
NearestNeighbors k=10
Uses Pillar-1 phone_table
No separate model file
Outputs (renormalised to 100%):
Age: 18-24/25-34/35-44/45-54/55+
Income: Low/LowerMid/Mid/High
Gender: Male/Female/NonBinary
RFM: Champion/Loyal/AtRisk/Hiber.

Pillar 4 — PMF Classifier
src/pillar4_pmf.py
— PMF CLASSIFIER —
RandomForestClassifier
Label: bottom 25% pmf = failure
PMF formula (6 signals, weighted):
NPS(25%) + Repeat(20%) + Rec(20%)
+ Return(15%) + Warranty(10%)
+ Sales_slope(10%)
Output: pmf_score, failure_risk%
Accuracy ~0.79 AUC ~0.88

MODEL STORE — /models/

joblib serialized · 18.9 MB total · 0 GPU · CPU-only

- **pillar1_segment.joblib** — 878 KB — XGBoost Classifier (Segment Predictor)
- **pillar1_similarity.joblib** — 2.7 MB — k-NN + phone_table (Similarity Engine)
- **pillar2_demand.joblib** — 9.7 MB — XGBoost Regressor (Demand Forecaster)
- **pillar4_pmf.joblib** — 5.2 MB — Random Forest (PMF Classifier)

trains & saves

EXTERNAL

search queries

Web Search API

DuckDuckGo Search API
ddgs library
max 8 parallel threads
text() + news() endpoints

DATA LAYER

CSV · 44,000 rows · 45 MB

global_smartphone_bi_v4_final.csv
44,000 rows · 45 MB · global smartphone dataset
22 launch specs (NUM_SPECS + CAT_SPECS) + POST_LAUNCH outcomes:
NPS · repeat_rate · return_rate · units_sold · social_mentions · demographics

reads CSV

TRAINING PIPELINE

Offline — run once with: python src/train_all.py

src/train_all.py
GroupShuffleSplit(test_size=0.20, random_state=42)
OrdinalEncoder (cat specs) · StandardScaler (similarity feats)
Trains: P1A Classifier · P1B k-NN · P2 Regressor · P4 PMF -> saves .joblib